Experiment 8 - Autoencoders and Variational Autoencoders

Dataset: Fashion-MNIST
Framework: PyTorch

```
!pip install wandb huggingface_hub -q

#url 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
```

```
import wandb
wandb.login()
```

```
True
```

```
from huggingface_hub import notebook_login
notebook_login()
```

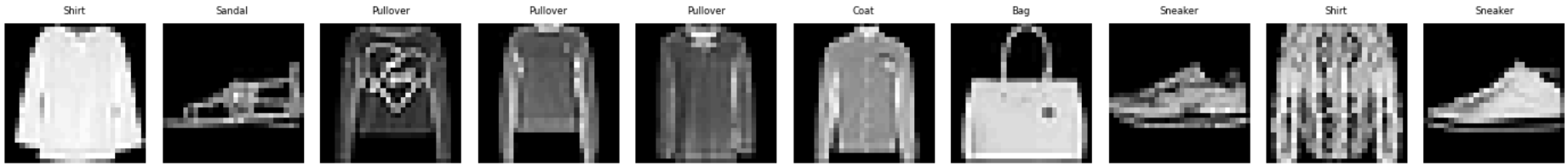
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import wandb
from torch.utils.data import DataLoader, random_split
from huggingface_hub import HfApi, create_repo

# change this to your huggingface username
HF_REPO = "mayank2424/exp8-ae-vae"

WANDB_PROJECT = "exp8-ae-vae"
EPOCHS       = 5
BATCH_SIZE   = 128
```

```
LATENT_DIMS  = [2, 8, 16, 32]
OPTIMIZERS   = ["adam", "sgd", "rmsprop"]
LOSS_FNS     = ["bce", "mse"]
DEVICE       = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(DEVICE)
```

```
cuda
```

```
transform = transforms.Compose([
    transforms.ToTensor()
])

full_train = torchvision.datasets.FashionMNIST(root='./data', train=True,  download=True, transform=transform)
test_data  = torchvision.datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)

n_train = int(0.8 * len(full_train))
n_val   = len(full_train) - n_train

train_set, val_set = random_split(full_train, [n_train, n_val], generator=torch.Generator().manual_seed(42))

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,  num_workers=2)
val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader  = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

print(f'train: {len(train_set)}  val: {len(val_set)}  test: {len(test_data)}')

class_names = ['T-shirt','Trouser','Pullover','Dress','Coat','Sandal','Shirt','Sneaker','Bag','Ankle boot']

imgs, labels = next(iter(train_loader))
fig, axes = plt.subplots(1, 10, figsize=(14, 2))
for i in range(10):
    axes[i].imshow(imgs[i].squeeze(), cmap='gray')
    axes[i].set_title(class_names[labels[i]], fontsize=6)
    axes[i].axis('off')
plt.tight_layout()
plt.show()
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 13.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 208kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.89MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 19.8MB/s]train: 48000  val: 12000  test: 10000
```



Part A - Autoencoder

```
class Autoencoder(nn.Module):
    def __init__(self, latent_dim):
        super(Autoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, latent_dim)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 784),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
```

```
        out = self.decoder(z).view(-1, 1, 28, 28)
        return out, z
```

Part B - Variational Autoencoder

```
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.latent_dim = latent_dim

        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU()
        )

        self.fc_mu     = nn.Linear(256, latent_dim)
        self.fc_logvar = nn.Linear(256, latent_dim)

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 784),
            nn.Sigmoid()
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + std * eps
```

```
    def decode(self, z):
        return self.decoder(z).view(-1, 1, 28, 28)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar, z
```

```
def recon_loss(recon, x, loss_type):
    if loss_type == 'bce':
        return nn.BCELoss()(recon, x)
    return nn.MSELoss()(recon, x)

def vae_loss(recon, x, mu, logvar, loss_type):
    r_loss = recon_loss(recon, x, loss_type)
    kl     = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())
    return r_loss + kl, r_loss, kl

def get_optimizer(model, name):
    if name == 'adam':
        return optim.Adam(model.parameters(), lr=1e-3)
    elif name == 'sgd':
        return optim.SGD(model.parameters(), lr=1e-3, momentum=0.9)
    else:
        return optim.RMSprop(model.parameters(), lr=1e-3)
```

```
def train_ae(model, loader, optimizer, loss_type):
    model.train()
    running = 0
    for x, _ in loader:
        x = x.to(DEVICE)
        optimizer.zero_grad()
        out, _ = model(x)
        loss = recon_loss(out, x, loss_type)
        loss.backward()
        optimizer.step()
        running += loss.item()
    return running / len(loader)
```

```
def evaluate_ae(model, loader, loss_type):
    model.eval()
    running = 0
    with torch.no_grad():
        for x, _ in loader:
            x = x.to(DEVICE)
            out, _ = model(x)
            running += recon_loss(out, x, loss_type).item()
    return running / len(loader)

def train_vae(model, loader, optimizer, loss_type):
    model.train()
    t_loss = t_recon = t_kl = 0
    for x, _ in loader:
        x = x.to(DEVICE)
        optimizer.zero_grad()
        out, mu, logvar, _ = model(x)
        loss, rl, kl = vae_loss(out, x, mu, logvar, loss_type)
        loss.backward()
        optimizer.step()
        t_loss += loss.item()
        t_recon += rl.item()
        t_kl += kl.item()
    n = len(loader)
    return t_loss/n, t_recon/n, t_kl/n

def evaluate_vae(model, loader, loss_type):
    model.eval()
    t_loss = t_recon = t_kl = 0
    with torch.no_grad():
        for x, _ in loader:
            x = x.to(DEVICE)
            out, mu, logvar, _ = model(x)
            loss, rl, kl = vae_loss(out, x, mu, logvar, loss_type)
            t_loss += loss.item()
            t_recon += rl.item()
            t_kl += kl.item()
    n = len(loader)
    return t_loss/n, t_recon/n, t_kl/n
```

```
def show_reconstructions(model, loader, model_type, title, n=8):
    model.eval()
    x, _ = next(iter(loader))
    x = x[:n].to(DEVICE)
    with torch.no_grad():
        out = model(x)[0]
    fig, axes = plt.subplots(2, n, figsize=(n * 1.4, 3))
    for i in range(n):
        axes[0, i].imshow(x[i].cpu().squeeze(), cmap='gray')
        axes[1, i].imshow(out[i].cpu().squeeze(), cmap='gray')
        axes[0, i].axis('off')
        axes[1, i].axis('off')
    axes[0, 0].set_ylabel('Input', fontsize=8)
    axes[1, 0].set_ylabel('Recon', fontsize=8)
    plt.suptitle(title, fontsize=9)
    plt.tight_layout()
    plt.show()
    return fig

def show_latent_2d(model, loader, model_type, title):
    model.eval()
    zs, ys = [], []
    with torch.no_grad():
        for x, y in loader:
            x = x.to(DEVICE)
            z = model(x)[1] if model_type == 'AE' else model.encode(x)[0]
            zs.append(z.cpu().numpy())
            ys.append(y.numpy())
    zs = np.concatenate(zs)
    ys = np.concatenate(ys)
    fig, ax = plt.subplots(figsize=(7, 5))
    sc = ax.scatter(zs[:, 0], zs[:, 1], c=ys, cmap='tab10', s=2, alpha=0.5)
    cb = plt.colorbar(sc, ax=ax)
    cb.set_ticks(range(10))
    cb.set_ticklabels(class_names)
    ax.set_title(title)
    ax.set_xlabel('z0')
    ax.set_ylabel('z1')
    plt.tight_layout()
```

```
    plt.show()
    return fig

def show_interpolation(model, loader, model_type, title, steps=10):
    model.eval()
    x, _ = next(iter(loader))
    x1 = x[0:1].to(DEVICE)
    x2 = x[1:2].to(DEVICE)
    with torch.no_grad():
        z1 = model(x1)[1] if model_type == 'AE' else model.encode(x1)[0]
        z2 = model(x2)[1] if model_type == 'AE' else model.encode(x2)[0]
    fig, axes = plt.subplots(1, steps, figsize=(steps * 1.3, 1.8))
    for i, a in enumerate(np.linspace(0, 1, steps)):
        z = (1 - a) * z1 + a * z2
        with torch.no_grad():
            img = model.decoder(z).view(28, 28) if model_type == 'AE' else model.decode(z).squeeze()
        axes[i].imshow(img.cpu(), cmap='gray')
        axes[i].set_title(f'{a:.1f}', fontsize=6)
        axes[i].axis('off')
    plt.suptitle(title, fontsize=9)
    plt.tight_layout()
    plt.show()
    return fig

def show_generated(model, title, n=10):
    model.eval()
    with torch.no_grad():
        z = torch.randn(n, model.latent_dim).to(DEVICE)
        imgs = model.decode(z)
    fig, axes = plt.subplots(1, n, figsize=(n * 1.3, 1.8))
    for i in range(n):
        axes[i].imshow(imgs[i].cpu().squeeze(), cmap='gray')
        axes[i].axis('off')
    plt.suptitle(title, fontsize=9)
    plt.tight_layout()
    plt.show()
    return fig
```

```
results = {}
trained_models = {}
```

```
def run_experiment(model_type, latent_dim, opt_name, loss_type):
    name = f'{model_type}_ld{latent_dim}_{opt_name}_{loss_type}'
    print(f'\nstarting: {name}')

    wandb.init(project=WANDB_PROJECT, name=name, config={
        'model': model_type,
        'latent_dim': latent_dim,
        'optimizer': opt_name,
        'loss': loss_type,
        'epochs': EPOCHS
    })

    model     = (Autoencoder(latent_dim) if model_type == 'AE' else VAE(latent_dim)).to(DEVICE)
    optimizer = get_optimizer(model, opt_name)

    for epoch in range(1, EPOCHS + 1):
        if model_type == 'AE':
            tr = train_ae(model, train_loader, optimizer, loss_type)
            vl = evaluate_ae(model, val_loader, loss_type)
            wandb.log({'epoch': epoch, 'train_loss': tr, 'val_loss': vl})
            print(f'  epoch {epoch}/{EPOCHS}  train={tr:.4f}  val={vl:.4f}')
        else:
            tr, tr_r, tr_k = train_vae(model, train_loader, optimizer, loss_type)
            vl, vl_r, vl_k = evaluate_vae(model, val_loader, loss_type)
            wandb.log({
                'epoch': epoch,
                'train_loss': tr, 'train_recon': tr_r, 'train_kl': tr_k,
                'val_loss':   vl, 'val_recon':   vl_r, 'val_kl':   vl_k
            })
            print(f'  epoch {epoch}/{EPOCHS}  train={tr:.4f}  recon={tr_r:.4f}  kl={tr_k:.4f}  val={vl:.4f}')

    if model_type == 'AE':
        test_loss = evaluate_ae(model, test_loader, loss_type)
    else:
        test_loss, _, _ = evaluate_vae(model, test_loader, loss_type)

    print(f'  test_loss={test_loss:.4f}')
    wandb.log({'test_loss': test_loss})
```

```
        fig = show_reconstructions(model, test_loader, model_type, name)
        wandb.log({'reconstructions': wandb.Image(fig)})
        plt.close(fig)

        fig = show_interpolation(model, test_loader, model_type, f'interpolation - {name}')
        wandb.log({'interpolation': wandb.Image(fig)})
        plt.close(fig)

        if latent_dim == 2:
            fig = show_latent_2d(model, test_loader, model_type, f'latent 2D - {name}')
            wandb.log({'latent_2d': wandb.Image(fig)})
            plt.close(fig)

        if model_type == 'VAE':
            fig = show_generated(model, f'generated - {name}')
            wandb.log({'generated': wandb.Image(fig)})
            plt.close(fig)

        torch.save(model.state_dict(), f'{name}.pt')
        trained_models[name] = model
        results[name] = test_loss

        wandb.finish()
        return model
```

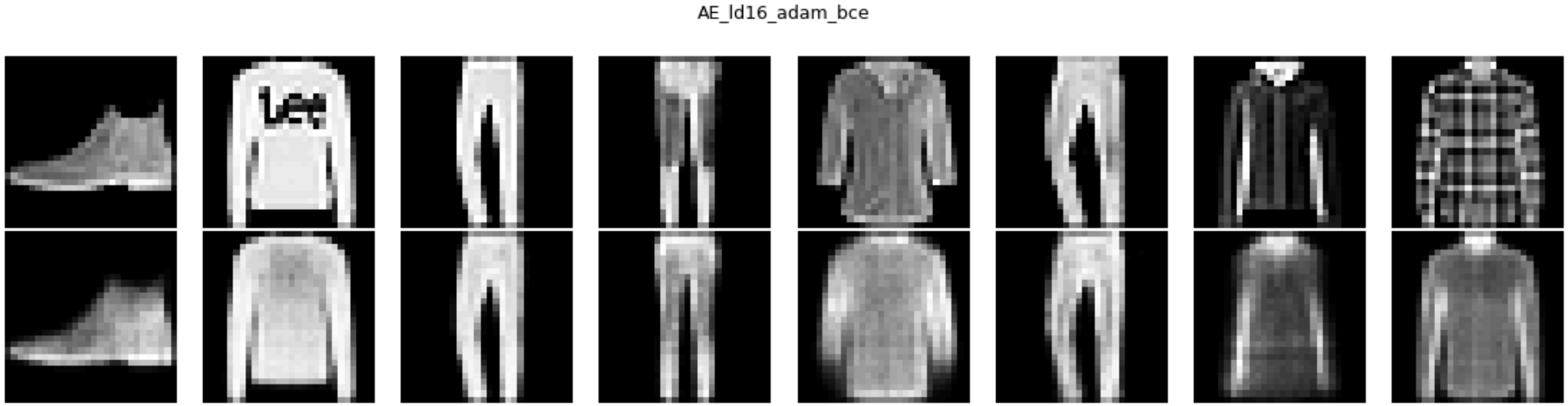
Quick Test - run this first

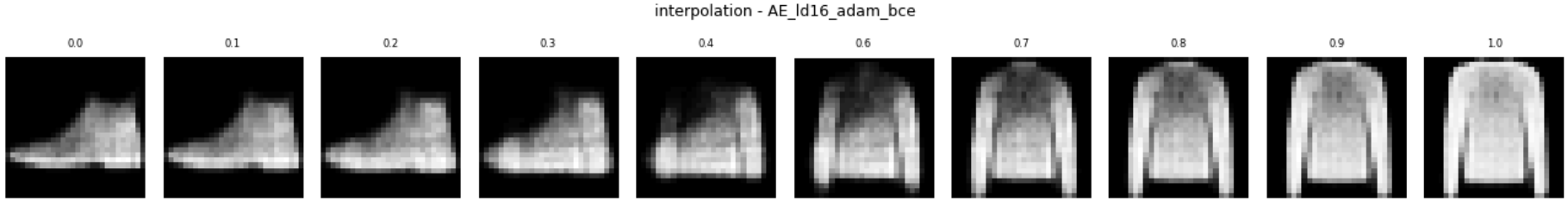
```
run_experiment('AE',  latent_dim=16, opt_name='adam', loss_type='bce')
run_experiment('VAE', latent_dim=16, opt_name='adam', loss_type='bce')
```

```
starting: AE_ld16_adam_bce
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_075343-eg7ydjw8`
Syncing run **AE_ld16_adam_bce** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae/runs/eg7ydjw8

```
  epoch 1/5  train=0.3451  val=0.3054
  epoch 2/5  train=0.2982  val=0.2936
  epoch 3/5  train=0.2900  val=0.2882
  epoch 4/5  train=0.2857  val=0.2850
  epoch 5/5  train=0.2827  val=0.2818
  test_loss=0.2833
```





Run history:

epoch

test_loss _

train_loss
val_loss

Run summary:

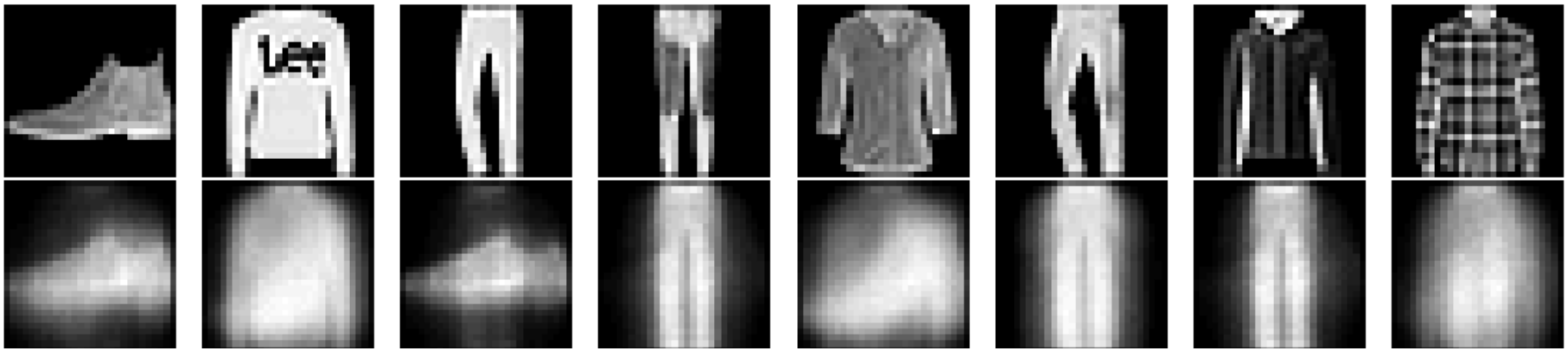
epoch	5
test_loss	0.28331
train_loss	0.28273
val_loss	0.28183

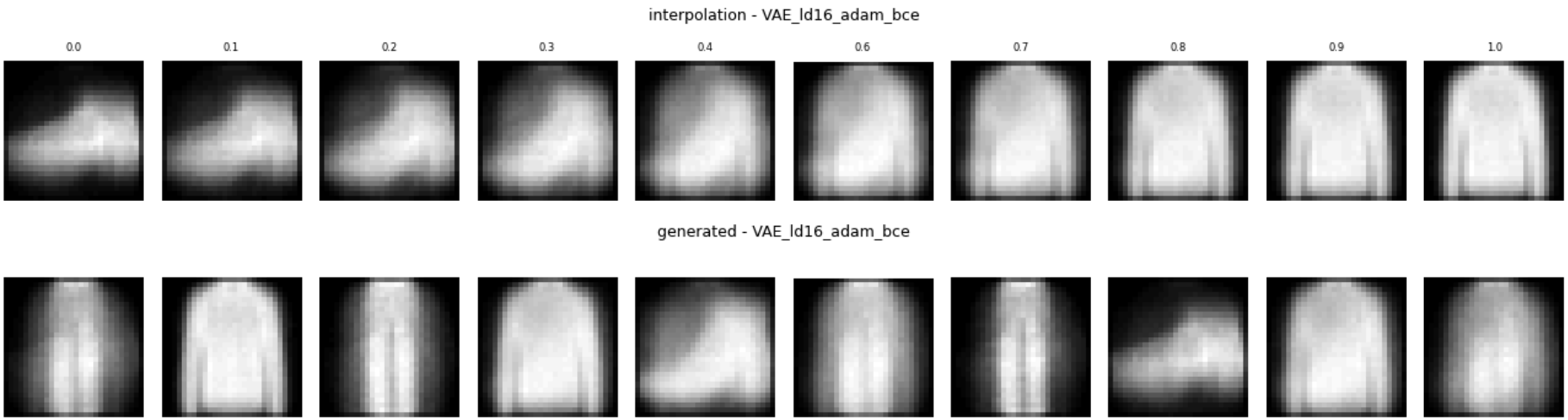
View run **AE_ld16_adam_bce** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae/runs/eg7ydjw8
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae
Synced 5 W&B file(s), 2 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_075343-eg7ydjw8/logs`

```
starting: VAE_ld16_adam_bce
```
Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_075431-4wotz88z`
Syncing run **VAE_ld16_adam_bce** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae/runs/4wotz88z

```
  epoch 1/5  train=0.4933  recon=0.4852  kl=0.0081  val=0.4863
  epoch 2/5  train=0.4804  recon=0.4490  kl=0.0315  val=0.4803
  epoch 3/5  train=0.4793  recon=0.4423  kl=0.0370  val=0.4795
  epoch 4/5  train=0.4779  recon=0.4355  kl=0.0424  val=0.4764
  epoch 5/5  train=0.4762  recon=0.4297  kl=0.0465  val=0.4771
  test_loss=0.4772
```

VAE_ld16_adam_bce





Run history:

epoch	▁▂▃▅▇
test_loss	▁
train_kl	▁▂▃▅▇
train_loss	▇▅▃▂▁▁
train_recon	▇▅▃▂▁▁
val_kl	▁▂▅▇▇
val_loss	▇▅▃▂▁▂
val_recon	▇▅▃▂▁▁

Run summary:

epoch	5
test_loss	0.47723
train_kl	0.0465
train_loss	0.47615
train_recon	0.42965
val_kl	0.04445
val_loss	0.47714

val_recon 0.43269

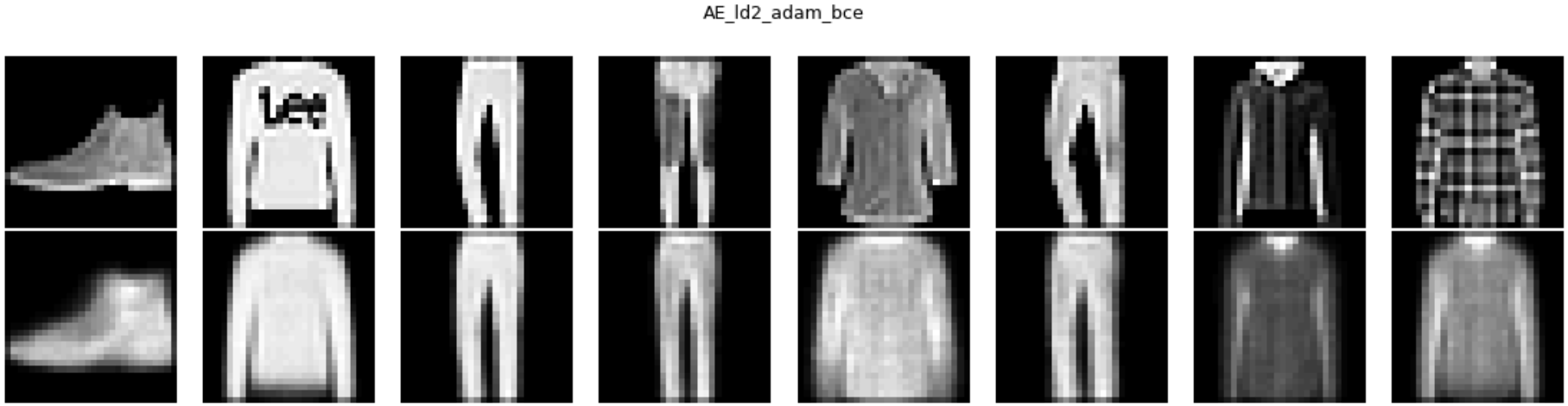
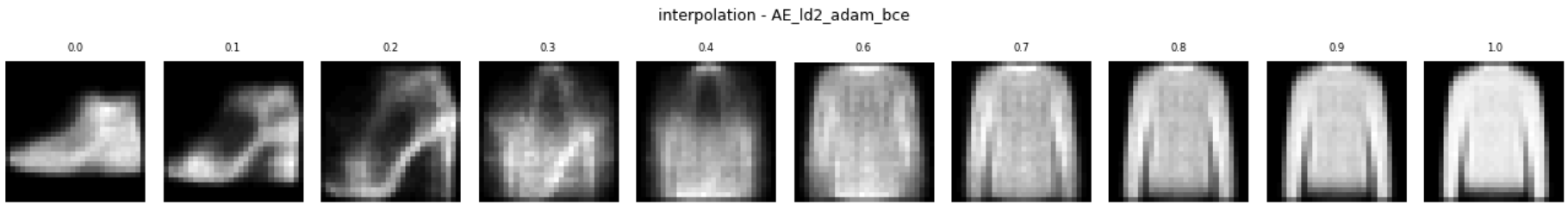
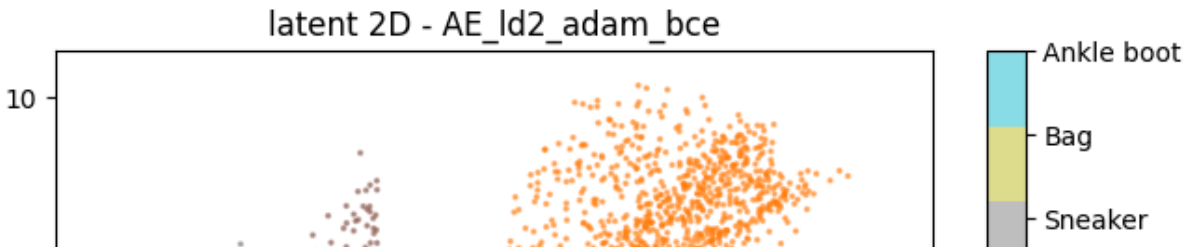
Full Grid - all 48 combinations

View run **VAE_ld16_adam_bce** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae/runs/4wotz88z

```
for model_type in ['AE', 'VAE']:
    for ld in LATENT_DIMS:
        for opt in OPTIMIZERS:
            for lf in LOSS_FNS:
                run_experiment(model_type, ld, opt, lf)
```

```
    (2): ReLU()
    (3): Linear(in_features=512, out_features=256, bias=True)
    (4): ReLU()
  )
  (fc_mu): Linear(in_features=256, out_features=16, bias=True)
  (fc_logvar): Linear(in_features=256, out_features=16, bias=True)
  (decoder): Sequential(
    (0): Linear(in_features=16, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=784, bias=True)
    (5): Sigmoid()
  )
)
```

```
starting: AE_ld2_adam_bce
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_075520-xltqmtik`
Syncing run **AE_ld2_adam_bce** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/exp8-ae-vae/runs/xltqmtik

```
  epoch 1/5  train=0.3578  val=0.3328
  epoch 2/5  train=0.3295  val=0.3280
  epoch 3/5  train=0.3262  val=0.3260
  epoch 4/5  train=0.3239  val=0.3245
  epoch 5/5  train=0.3223  val=0.3226
  test_loss=0.3240
```